ARCHITECTURE REVIEW

FormID Database Manager

Reviewed	2026-07-21 16:00 PDT
Worktree	`architecture @ 379f234`
Basis	code + ADR + GitHub

module · seam · leakage · deep module

GUARDRAIL

ADR-0001 protects Store depth

ROADMAP

#2 left three follow-on seams

LIVE GITHUB

#10–#15 are completed

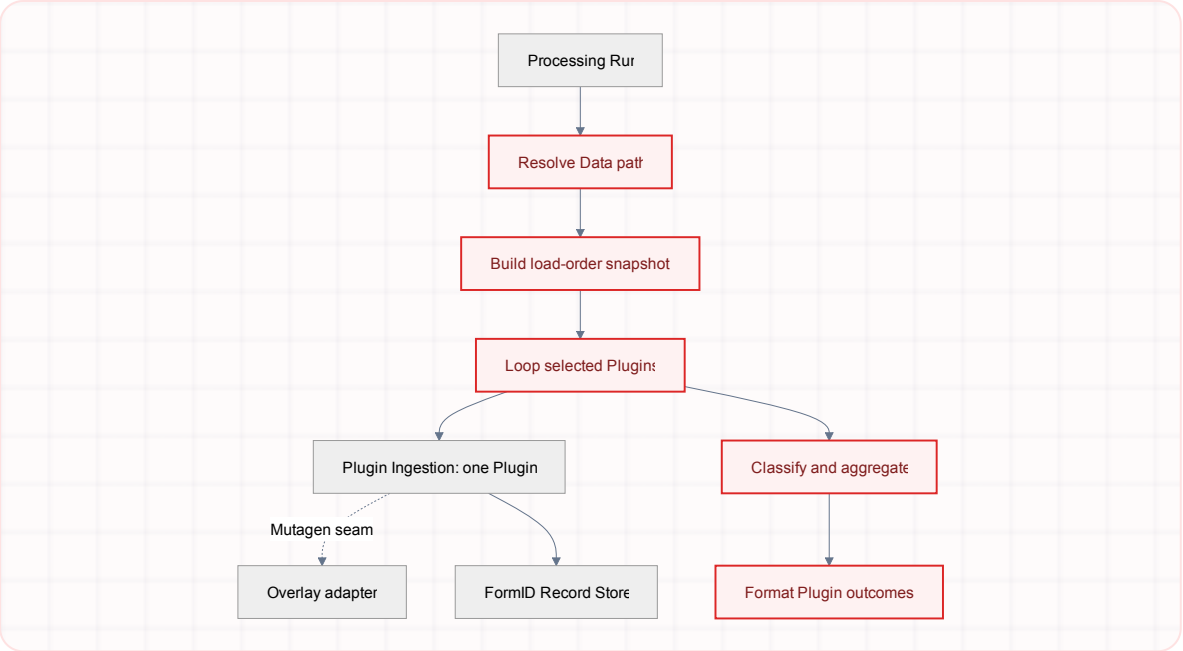
CANDIDATE 01

Deepen Plugin Ingestion across selected Plugins

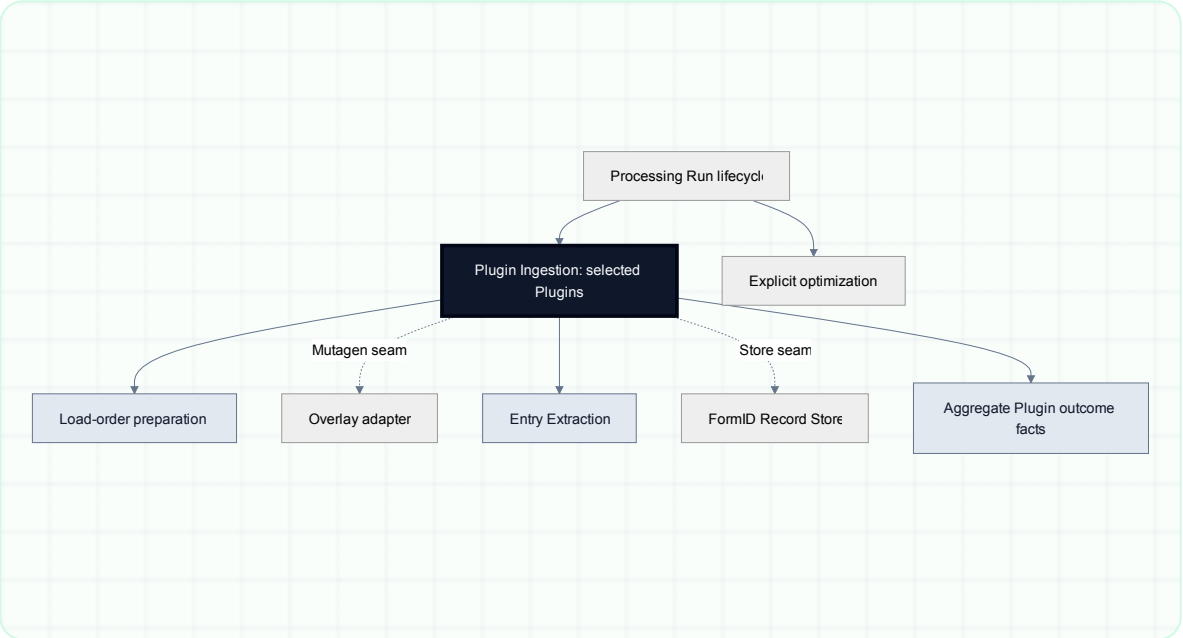
Strong · mock

`Core/Services/ProcessingRun.cs · 516–650`
`Core/Services/PluginIngestion.cs · 15–20, 63–159`
`Tests/Unit/Services/ProcessingRunExecutorTests.cs · 850–890`
`Tests/Unit/Services/PluginIngestionTests.cs`

BEFORE · DOMAIN PHASE LEAKS INTO CALLER



AFTER · DOMAIN-SIZED DEEP MODULE



PROBLEM

Processing Run owns load-order preparation, the selected-Plugin loop, Plugin result classification, counts, and formatting even though those are Plugin Ingestion implementation.

SOLUTION

Let one deep Plugin Ingestion module own the selected Plugin set through aggregate Plugin outcome facts while Processing Run keeps Store opening, cancellation, explicit optimization, terminal reporting, and cleanup.

WINS

locality: selected-Plugin rules stay together	leverage: caller learns one phase

Mutagen details stop leaking	tests hit one interface

> **Deletion test:** the selected-Plugin mechanics disappear from Processing Run and concentrate behind Plugin Ingestion; they do not reappear across callers.

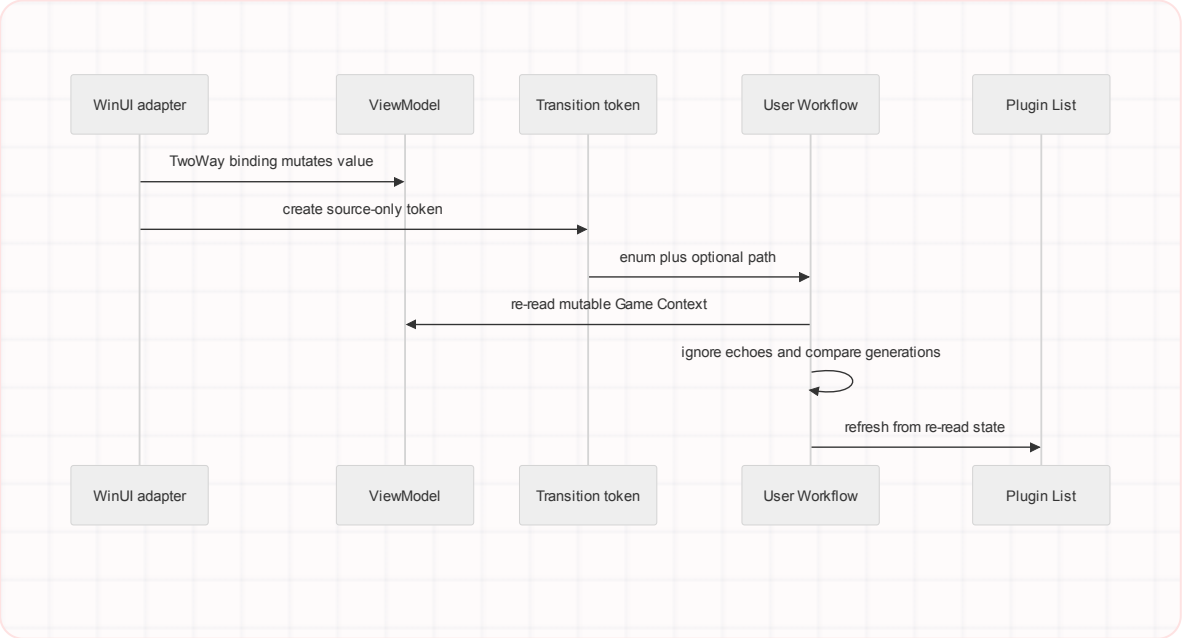
CANDIDATE 02

Make Game Context value-owning

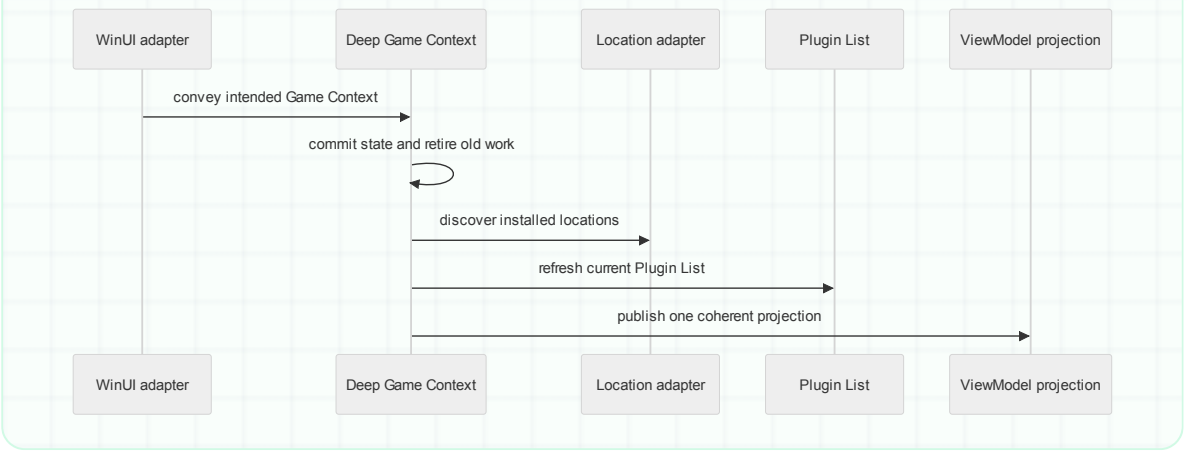
Strong · local-substitutable

```
WinUI/MainWindow.xaml · 50–51, 83–84, 252
WinUI/MainWindow.xaml.cs · 106–149
Core/Services/GameContextTransition.cs · 3–57
Core/Services/UserWorkflow.cs · 20–22, 55–69, 205–367
```

BEFORE · SOURCE TOKEN AFTER MUTATION



AFTER · ONE OWNER OF INTENDED VALUES



PROBLEM

WinUI mutates bound state before sending a source-only transition, so User Workflow correctness depends on re-reading mutable projection state, suppressing programmatic echoes, and coordinating generations.

SOLUTION

Deepen Game Context inside User Workflow so intended values, committed state, installed-location choice, projection, and obsolete-work retirement share one implementation.

WINS

- locality: ordering rules become internal
- interface carries intended Game Context
- stale work retires once
- tests stop pre-mutating state

> **Deletion test:** deleting the transition enum, factories, and routing switch removes interface surface without moving meaningful implementation.

Roadmap fit: closed issue #2 deliberately left Game Context deepening for a separate proposal.

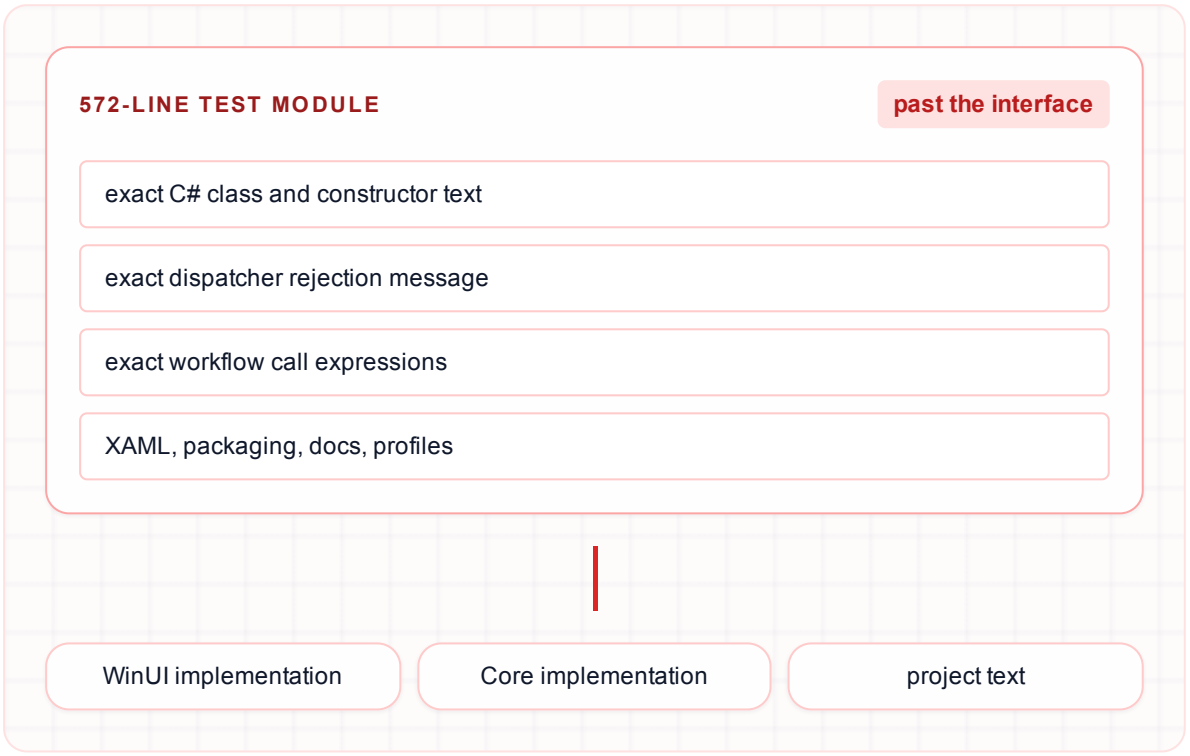
CANDIDATE 03

Verify WinUI adapters through real interfaces

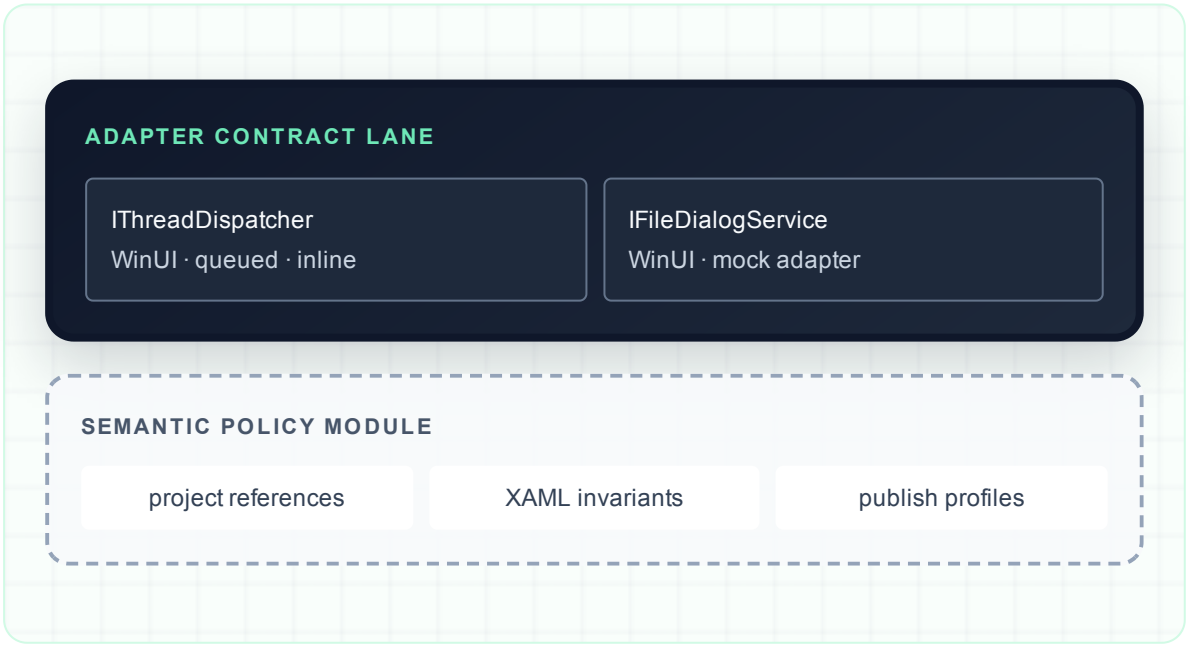
Worth exploring | local-substitutable

```
Tests/Unit/Architecture/WinUiPlatformServiceSourceTests.cs · 15–169
Tests/FormID Database Manager.Tests.csproj · 4, 31–34
WinUI/FormID Database Manager.WinUI.csproj · 5, 39–41
Core/Services/IThreadDispatcher.cs · 28–64
TestUtilities/Mocks/SynchronousThreadDispatcher.cs · 11–31
```

BEFORE · ONE SOURCE SNAPSHOT REACHES EVERYWHERE



AFTER · REPLACE WITH TWO FOCUSED TEST SURFACES



PROBLEM

A plain .NET test module cannot reference the Windows-targeted WinUI module, so 572 lines verify implementation text, exact expressions, and concrete adapter arrangement rather than behavior at the seams.

SOLUTION

Replace superseded C# text assertions with a Windows-targeted adapter contract lane and a small semantic policy module, while keeping declarative XAML and publish invariants.

WINS

- tests observe adapter behavior
- renames stop breaking verification
- semantic policies gain locality
- declarative XAML checks remain

> **Deletion test:** harmless implementation rearrangement fails today, proving the source snapshot crosses past the interface; replacement contract tests preserve behavior without preserving spelling.

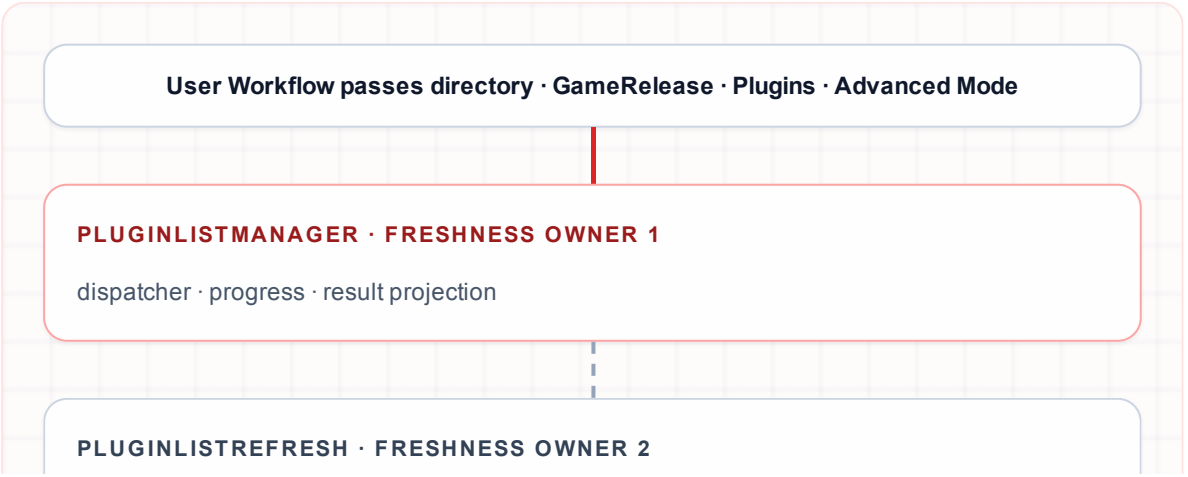
CANDIDATE 04

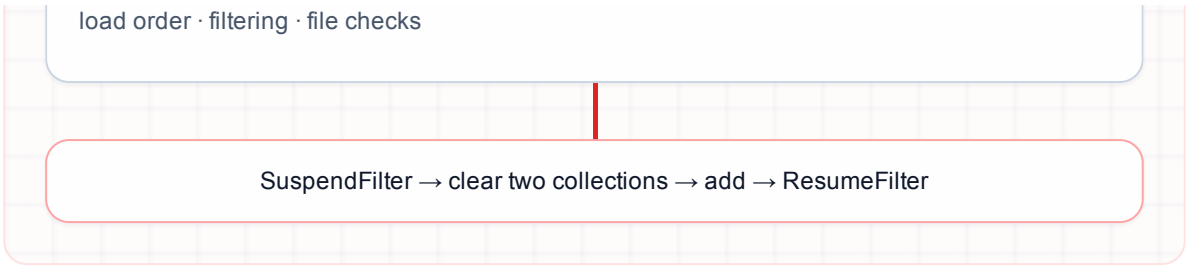
Give Plugin List projection one owner

Worth exploring | local-substitutable

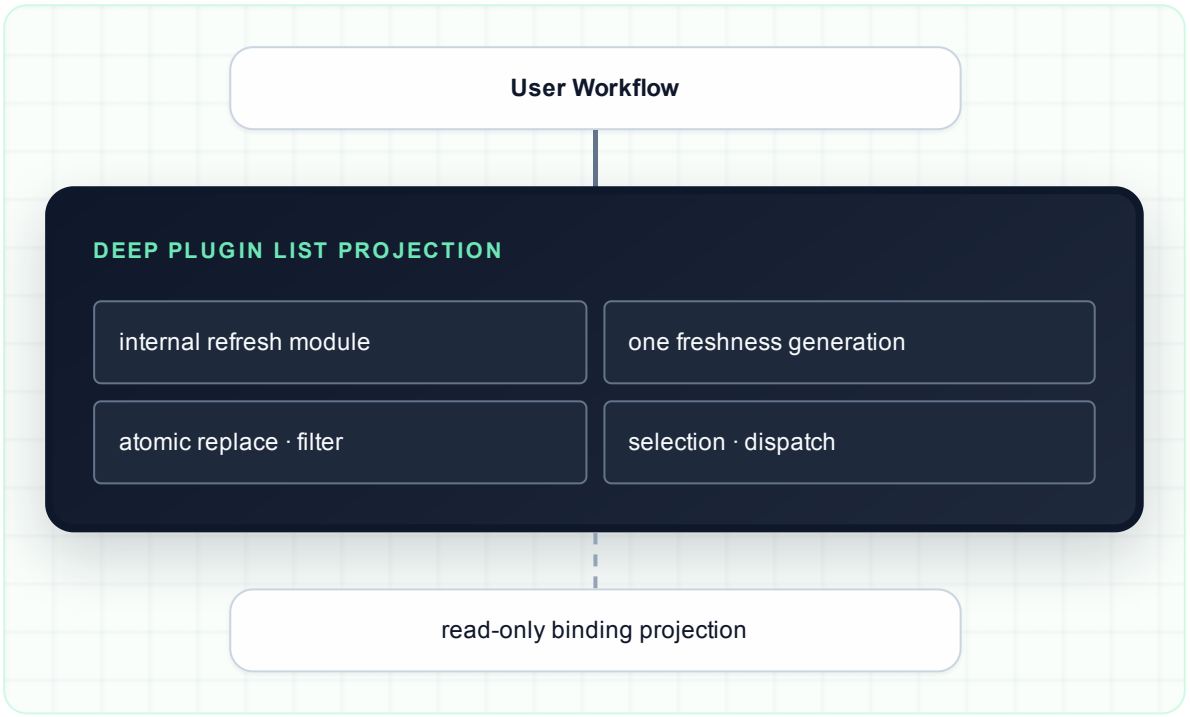
```
Core/Services/PluginListManager.cs · 13–16, 52–217
Core/Services/PluginListRefresh.cs · 5–10, 74–167
Core/ViewModels/MainWindowViewModel.cs · 121–154, 247–326
Core/Services/UserWorkflow.cs · 118, 126, 297–301
```

BEFORE · PROJECTION PROTOCOL CROSSES SEAMS





AFTER · ONE DEEP PROJECTION IMPLEMENTATION



PROBLEM

PluginListManager owns the ViewModel yet accepts its mutable collection, coordinates a public suspend/resume protocol, mutates two collections, and duplicates freshness ownership with PluginListRefresh.

SOLUTION

Let one deep Plugin List projection implementation own atomic replacement, filtering, selection, dispatch, and end-to-end freshness while retaining PluginListRefresh as an internal deep module.

WINS

- locality: one freshness owner
- interface hides collection mutation
- filters replace atomically
- tests cross one seam

> **Deletion test:** removing the projection protocol concentrates its implementation behind the Plugin List interface; it does not need to spread through User Workflow or the ViewModel.

Observed friction: commit 8e8aa9a added a second freshness check at dispatcher application after stale results could overwrite newer state.

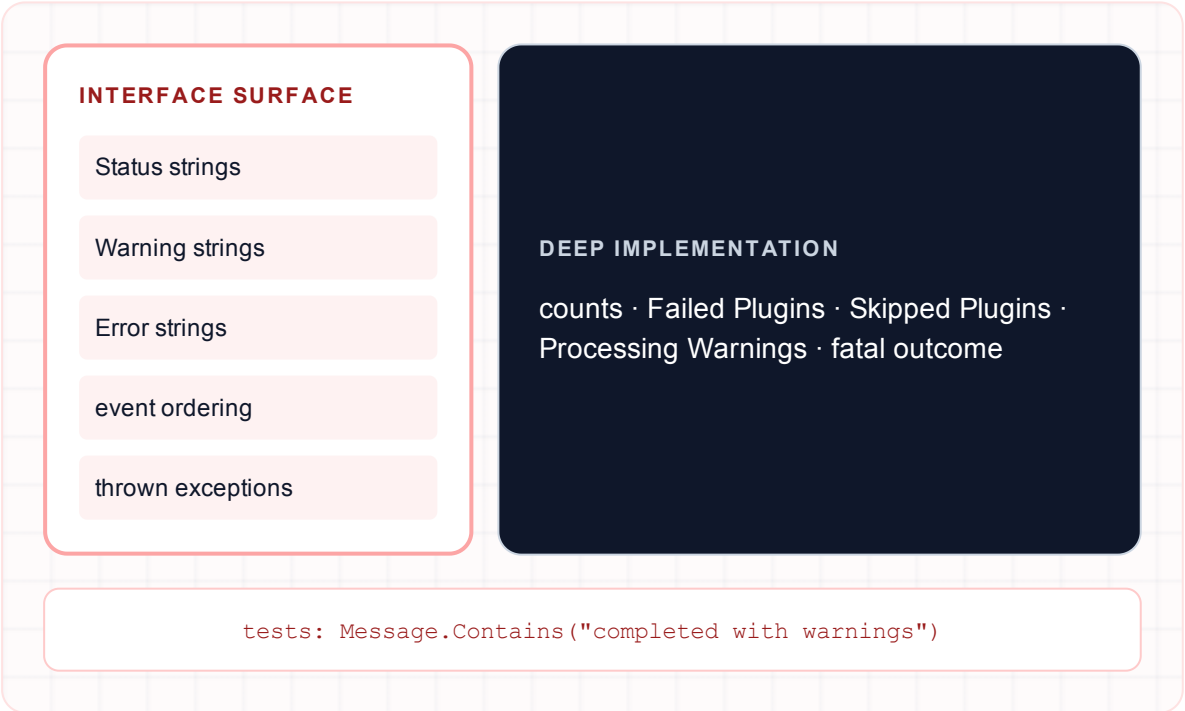
CANDIDATE 05

Make Processing Run outcomes authoritative

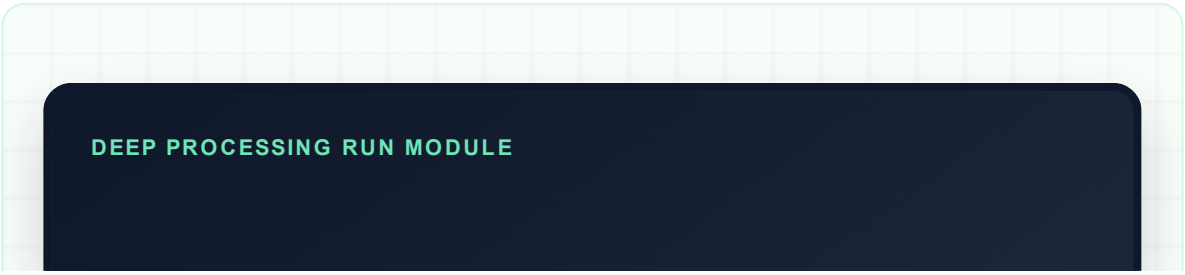
Worth exploring | in-process

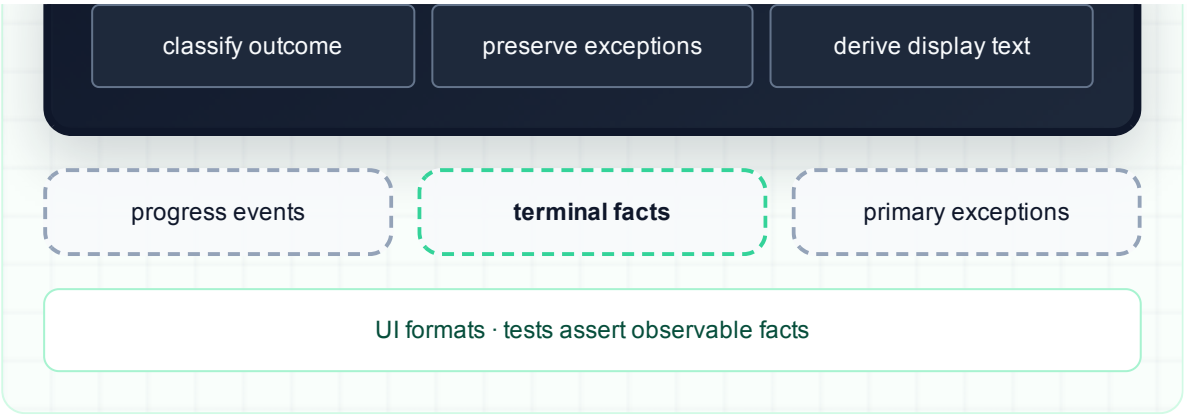
```
Core/Services/ProcessingRun.cs · 181–227, 455–463, 577–650
Core/Services/UserWorkflow.cs · 390–405
Tests/Unit/Services/ProcessingRunExecutorTests.cs
Tests/Integration/ProcessingRunIntegrationTests.cs · 103–112
```

BEFORE · TERMINAL MEANING HIDES IN TEXT



AFTER · SMALL AUTHORITATIVE OUTCOME SURFACE





PROBLEM

The domain defines five terminal Processing Run states, but callers and tests infer them from free-form messages, event ordering, event kind, and thrown exceptions.

SOLUTION

Keep progress reporting, but make terminal facts authoritative inside the deep Processing Run module and derive user-visible text at the presentation adapter.

WINS

- terminal states become explicit
- tests stop parsing messages
- progress loses hidden contract
- lifecycle exceptions stay primary

> **Deletion test:** ProcessingRunEvent earns its seam, but the hidden terminal-message convention does not; removing literal formatting should not remove terminal classification.

ADR-0001 guardrail: optimization stays explicit, optimization failures stay visible, cancellation still throws, and Store disposal remains best-effort cleanup.

Protected depth

FORMID RECORD STORE

ADR-0001 fixes ready-on-return opening, sole SQLite ownership, and explicit optimization.

PROCESSING RUN LIFECYCLE

The small execution interface hides cancellation ownership, Store cleanup, and optimization ordering.

REAL SEAMS

Dispatcher, file dialog, load order, Mutagen overlay, and Store adapters each have justified alternatives.

INTERNAL DEPTH

PluginListRefresh and Entry Extraction already hide substantial implementation behind small interfaces.

Small deletion-test wins

- Remove the unused internal MainWindow migration constructor and its source assertion.
- Prune IGameLoadOrderProvider.GetListedPluginNames; production has no caller.
- Replace the drifting SynchronousThreadDispatcher with the canonical inline adapter.

TOP RECOMMENDATION

Deepen Plugin Ingestion across selected Plugins

It matches the domain definition, removes Mutagen and selected-Plugin mechanics from Processing Run, and improves locality without reopening the completed FormID Record Store or Processing Run lifecycle decisions.